

CodeDNA

Un protocollo portatile nel sorgente per gli agenti AI.

Il file è il canale

Portatile con git

MIT · open source

La conoscenza muore tra una sessione e l'altra.

AGENTE A

Sistema il bug

Modifica **utils.py**. Non sa che 18 file lo importano. pubblica una rottura.

AGENTE B

Riscopre il vincolo

Una settimana dopo, venti minuti per ritrovare una regola che A aveva già capito e non aveva scritto.

AGENTE C

Salta il filtro

Chiama **get_invoices()** senza escludere i tenant sospesi. Il requisito stava in un altro file.

Ogni agente riparte da zero. Modello diverso, strumento diverso, sessione diversa.

Portatile al 100%, perché sta nel sorgente.

Chi clona il repository ha il contesto. Su qualsiasi macchina, con qualsiasi agente.

Viaggia con git

Il protocollo è nel file, accanto al codice che descrive. Un clone basta.

Qualunque agente

Altro modello, altro strumento, altra sessione. Legge le stesse note. Nessun fine-tuning.

Nessun servizio

Per usarlo non serve una memoria esterna, un database vettoriale o un account.

Quattro livelli, come uno zoom.

LIVELLO 0

.codedna

Mappa del progetto: pacchetti, dipendenze, file chiave. Si legge prima di aprire il codice.

LIVELLO 1

Header del modulo

exports, used_by, related, rules, message. Poche decine di token, in cima al file.

LIVELLO 2

Rules sulla funzione

Il vincolo sta sulla funzione che lo deve rispettare, non in un documento a parte.

LIVELLO 3

Rules inline

Una riga sopra la logica non ovvia: un if, un ordine di operazioni, un caso limite.

Un file basta per orientarsi.

```
"""revenue.py – Ricavi mensili.

exports: monthly_revenue(year, month) → dict
used_by: api/reports.py → revenue_route
         api/serializers.py → Schema [cascade]
related: billing/currency.py – multi-valuta
wiki:    docs/wiki/revenue.md
rules:   get_invoices() ritorna TUTTI i tenant
         filtrare is_suspended() PRIMA della somma
message: arrotondamento multi-valuta
"""
```

used_by — due file dipendono da questo. Uno è cascade: se cambio il contratto, va aggiornato.

rules — il filtro sui tenant sospesi va applicato prima della somma. C non può più saltarlo.

wiki — la pagina esiste. L'agente sa che l'header non gli basta.

Quattro domande, quattro risposte.

used_by:

Chi importa questo file. È il grafo inverso: l'agente lo segue invece di cercare in tutto il repository.

related:

Chi fa la stessa cosa senza importarmi. Il bug trasversale, senza una catena di import.

rules:

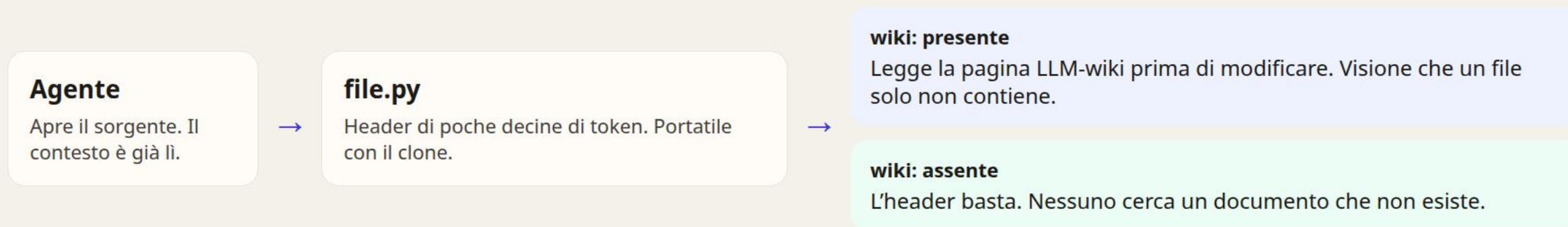
Il meccanismo, non la soluzione. «La conversione timezone avviene prima», non «modifica questi tre file».

message:

Nota per l'agente successivo. Se la conferma, diventa una rule. Se non torna, la scarta con un motivo.

La wiki collabora. Non è un prerequisito.

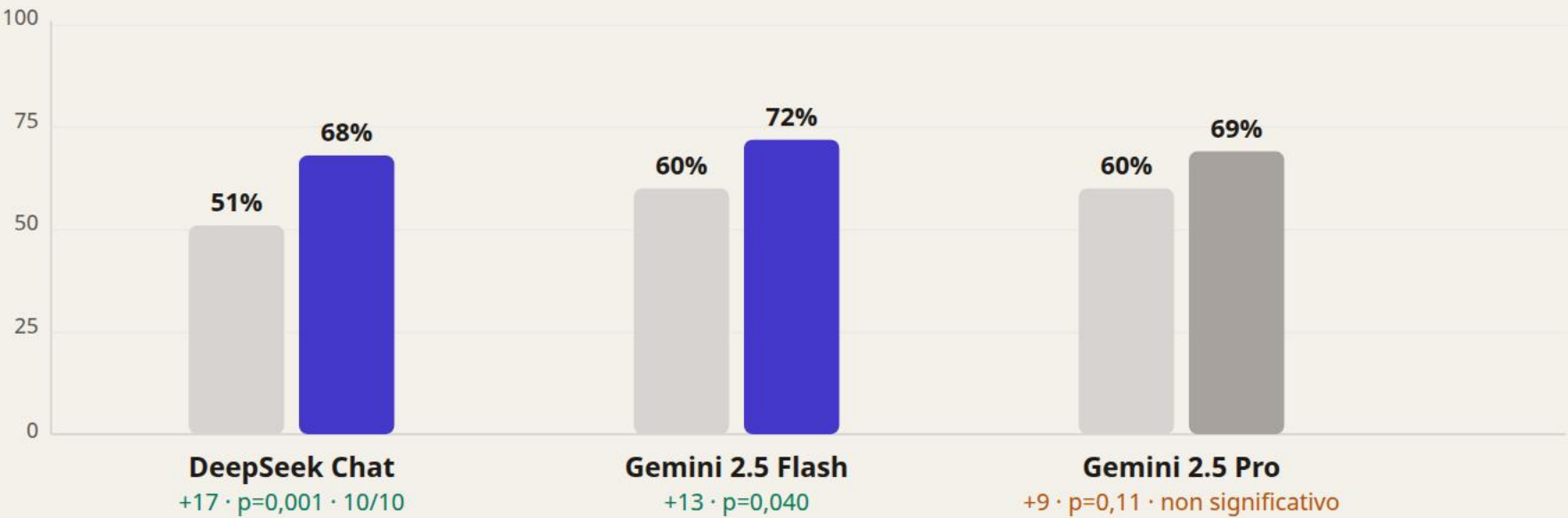
Le annotazioni restano corte. Quando una regola non ci entra, il campo wiki: punta al piano superiore.



CodeDNA genera già questa memoria, nel pattern LLM-wiki di Karpathy. Nel file la verità strutturale. Nella wiki il racconto più lungo. Si usano insieme.

Trova i file giusti.

Senza annotazioni Con CodeDNA F1 sui file aperti · unica differenza: le annotazioni



related: trova chi non ti importa.

Cinque file con la stessa logica IDNA. Nessuna catena di import li collega.

SENZA ANNOTAZIONI

2/5



F1 40%. Solo la cartella mail.

SOLO USED_BY:

2/5



F1 40%. Gli import non arrivano lì.

USED_BY: + RELATED:

5/5



F1 100%. Un caso riportato, non una media.

La regola dice l'ordine, non i file.

La conversione di timezone deve avvenire prima delle funzioni datetime.

SENZA CODEDNA

6/7

File allineati alla patch ufficiale. Cinque modifiche fallite. L'agente vede la funzione giusta e non la tocca.

CON CODEDNA

7/7

File allineati alla patch ufficiale. Nessuna modifica fallita.

Il codice trasporta il contratto.

RPG · DEEPSEEK CHAT

1,6×

1 h 59 min contro 3 h 11 min. Con CodeDNA il gioco è giocabile.
Senza, una scena statica: moduli dichiarati e mai scritti.

SAAS · DEEPSEEK R1

98%

File annotati senza promemoria a metà sessione. 83 min contro
99. Complessità ciclomatica 2,1 contro 3,1.

Il primo agente scrive `used_by`: anche per file che ancora non esistono.

Tre comandi, in locale.

```
pipx install git+https://github.com/Larens94/codedna.git  
codedna install --path . --tools codex --no-wiki-sync  
codedna init . --no-llm
```

1 • CLI

Python 3.11+. Installazione isolata, senza toccare l'ambiente globale.

2 • Strumento

Al posto di codex: claude, cursor, copilot, aider e gli altri della tabella.

3 • Annota

Pass strutturale gratuito: exports e used_by. Le rules semantiche sono un passo dopo, con un modello.

Quattro limiti, detti subito.

- Gemini 2.5 Pro guadagna meno e il risultato non è significativo ($p=0,11$).
- Il 5/5 di related: è un caso, il bug unicode. Non è la media del benchmark.
- Il 7/7 del #13495 è la qualità della patch. L'F1 di navigazione sullo stesso bug, con altri modelli, non racconta la stessa storia.
- Sono risultati riportati dal progetto. Le run grezze non sono nel repository.

Il contesto viaggia col sorgente.

CodeDNA lo tiene corto, nel file. La LLM wiki, se il puntatore c'è, tiene il resto. Insieme.

github.com/Larens94/codedna

MIT

DOI 10.5281/zenodo.19158336